

Name :- M.Sreekari

Roll no :- 2311CS040099

Dept. :- CS Beta

Subject :- Applied Agentic AI

1. Lab exp – 5 :- Multi-Agent SDR System

Develop a multi-agent AI system for sales development that generates potential leads, qualifies the leads based on predefined criteria, and creates personalized outreach emails for qualified leads.

Code :-

```
from groq import Groq
```

```
API_KEY = "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"
```

```
client = Groq(api_key=API_KEY)
```

```
def ask(prompt):
```

```
    response = client.chat.completions.create(
```

```
        model="openai/gpt-oss-120b",
```

```
        messages=[
```

```
            {
```

```
                "role": "user",
```

```
                "content": prompt
```

```
            }
```

```
        ]
```

```
    )
```

```
    return response.choices[0].message.content
```

```
# AGENT 1: LEAD GENERATION
```

```
def lead_generation_agent(product, industry):
```

```
    prompt = f"""
```

```
You are a Lead Generation Agent.
```

```
Your task is to identify potential customers
```

```
for the given product.
```

Product:

{product}

Target Industry:

{industry}

Generate 3 fictional example companies.

For each company provide:

Company Name:

Industry:

Company Size:

Potential Need:

Reason why this company could be a good lead:

Return the results as a numbered list.

"""

```
    return ask(prompt)
```

AGENT 2: LEAD QUALIFICATION

```
def qualification_agent(leads, product):
```

```
    prompt = f"""
```

You are a Lead Qualification Agent.

Your task is to evaluate potential customers

for the following product.

Product:

{product}

Potential Leads:

{leads}

For each lead:

1. Give a qualification score from 1 to 10.
2. Explain the reason for the score.
3. Decide whether the lead is QUALIFIED or NOT QUALIFIED.

Consider:

- Relevance of the industry
- Company size
- Potential need for the product
- Potential business value

Only leads with a score of 7 or higher

should be marked as QUALIFIED.

Return the results clearly.

"""

```
    return ask(prompt)
```

AGENT 3: EMAIL GENERATION

```
def email_agent(qualified_leads, product):
```

```
    prompt = f"""
```

You are an SDR Email Agent.

Your task is to create personalized outreach

emails for QUALIFIED leads.

Product:

{product}

Qualified Leads:

{qualified_leads}

For every QUALIFIED lead:

1. Create a short subject line.
2. Write a professional personalized email.
3. Explain how the product could help the company.
4. Include a simple call to action.
5. Keep the email under 120 words.

Do NOT create emails for leads marked

NOT QUALIFIED.

Return each email separately.

```
"""
```

```
    return ask(prompt)=
```

```
# MAIN SDR SYSTEM
```

```
def run_sdr_system():
```

```
    print("\n=====")
```

```
    print("    MULTI-AGENT SDR SYSTEM")
```

```
    print("=====")
```

```
    product = input(
```

```
        "\nEnter the product/service: "
```

```
)
```

```
    industry = input(
```

```
        "Enter the target industry: "
```

```
)
```

```
# LEAD GENERATION
```

```
print("\n=====")
```

```
print("    LEAD GENERATION AGENT")
```

```
print("=====")
```

```
leads = lead_generation_agent(
```

```
    product,
```

```
    industry
```

```
)
```

```
print("\nGenerated Leads:\n")
```

```
print(leads)
```

```
# LEAD QUALIFICATION
```

```
print("\n=====")
```

```
print("    LEAD QUALIFICATION AGENT")
```

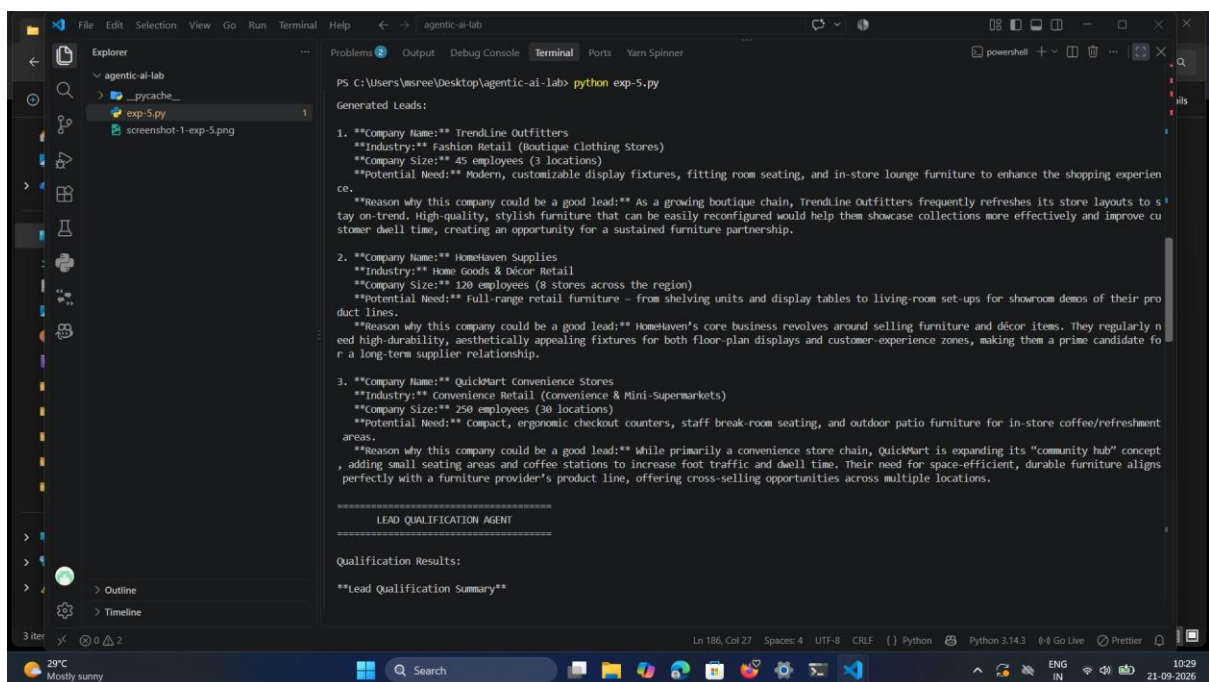
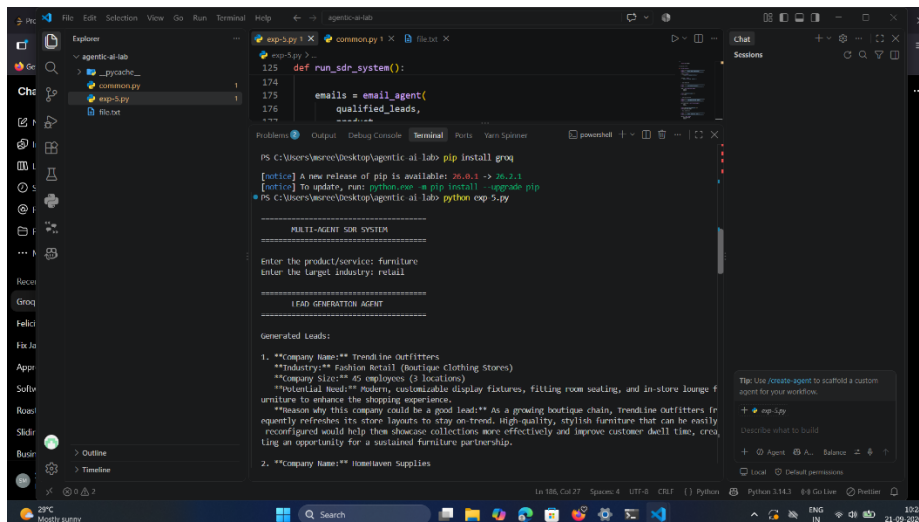
```
print("=====")
```

```

qualified_leads = qualification_agent(
    leads,
    product
)
print("\nQualification Results:\n")
print(qualified_leads)
# EMAIL GENERATION
print("\n=====")
print("    EMAIL GENERATION AGENT")
print("=====")
emails = email_agent(
    qualified_leads,
    product
)
print("\nGenerated Emails:\n")
print(emails)
# PROGRAM START
if __name__ == "__main__":
    run_sdr_system()

```

Output :-



2. Lab-exp-1 :- Text-to-SQL Workflow

Develop an end-to-end LLM workflow that converts natural-language questions into SQL queries, executes them on a database, and generates a natural-language response from the query results.

Code :-

```
import sqlite3
from groq import Groq
# GROQ API KEY
API_KEY = "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"
client = Groq(api_key=API_KEY)
# DATABASE
DATABASE = "company.db"
```

```

# CREATE SAMPLE DATABASE
def setup_database():
    connection = sqlite3.connect(DATABASE)
    cursor = connection.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS employees (
            id INTEGER PRIMARY KEY,
            name TEXT,
            department TEXT,
            salary INTEGER
        )
    """)
    cursor.execute("SELECT COUNT(*) FROM employees")
    count = cursor.fetchone()[0]
    if count == 0:
        employees = [
            (1, "Rahul", "Engineering", 70000),
            (2, "Priya", "HR", 55000),
            (3, "Arjun", "Engineering", 80000),
            (4, "Sneha", "Finance", 65000),
            (5, "Vikram", "Engineering", 75000)
        ]

        cursor.executemany("""
            INSERT INTO employees
            (id, name, department, salary)
            VALUES (?, ?, ?, ?)
        """, employees)

        connection.commit()
        connection.close()
# LLM FUNCTION
def ask_llm(prompt):
    response = client.chat.completions.create(
        model="openai/gpt-oss-120b",
        messages=[
            {
                "role": "user",
                "content": prompt
            }
        ]
    )

    return response.choices[0].message.content

```

```
# GET DATABASE SCHEMA
```

```
def get_schema():
```

```
    connection = sqlite3.connect(DATABASE)
    cursor = connection.cursor()
```

```
    cursor.execute("""
        SELECT name
        FROM sqlite_master
        WHERE type='table'
    """)
```

```
    tables = cursor.fetchall()
```

```
    schema = ""
```

```
    for table in tables:
```

```
        table_name = table[0]
```

```
        cursor.execute(
            f"PRAGMA table_info({table_name})"
        )
```

```
        columns = cursor.fetchall()
```

```
        schema += f"\nTable: {table_name}\n"
```

```
        for column in columns:
```

```
            column_name = column[1]
            column_type = column[2]
```

```
            schema += f"- {column_name} ({column_type})\n"
```

```
    connection.close()
```

```
    return schema
```

```
# TEXT TO SQL
```

```
def generate_sql(question, schema):
```

```
prompt = f"""
You are a Text-to-SQL Agent.
```

```
Convert the user's natural language question
into a valid SQLite SQL query.
```

```
Database Schema:
{schema}
```

```
User Question:
{question}
```

```
Rules:
```

1. Generate only a SQL SELECT query.
2. Do not use INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, or other modification commands.
3. Use only tables and columns that exist in the schema.
4. Return ONLY the SQL query.
5. Do not use markdown code fences.

```
"""
```

```
sql = ask_llm(prompt)
```

```
sql = sql.replace("```sql", "")
```

```
sql = sql.replace("```", "")
```

```
sql = sql.strip()
```

```
return sql
```

```
# =====
```

```
# VALIDATE SQL
```

```
# =====
```

```
def validate_sql(sql):
```

```
    sql_upper = sql.upper().strip()
```

```
    if not sql_upper.startswith("SELECT"):
        return False
```

```
    forbidden_keywords = [
        "INSERT",
```

```
"UPDATE",  
"DELETE",  
"DROP",  
"ALTER",  
"CREATE",  
"TRUNCATE",  
"REPLACE"  
]
```

```
for keyword in forbidden_keywords:
```

```
    if keyword in sql_upper:  
        return False
```

```
return True
```

```
# EXECUTE SQL
```

```
def execute_sql(sql):
```

```
    connection = sqlite3.connect(DATABASE)  
    cursor = connection.cursor()
```

```
    try:
```

```
        cursor.execute(sql)
```

```
        rows = cursor.fetchall()
```

```
        column_names = [  
            description[0]  
            for description in cursor.description  
        ]
```

```
        connection.close()
```

```
        return column_names, rows
```

```
    except Exception as error:
```

```
        connection.close()
```

```
        return None, str(error)
```

```
# GENERATE FINAL ANSWER
```

```
def generate_answer(question, columns, rows):
```

```
    data = ""
```

```
    for row in rows:
```

```
        data += str(row) + "\n"
```

```
    prompt = f"""
```

```
You are an Answer Generation Agent.
```

```
Answer the user's question using the SQL query results.
```

```
User Question:
```

```
{question}
```

```
Columns:
```

```
{columns}
```

```
Query Results:
```

```
{data}
```

```
Give a clear and concise natural-language answer.
```

```
Do not invent information that is not present  
in the query results.
```

```
"""
```

```
    return ask_llm(prompt)
```

```
# MAIN TEXT-TO-SQL WORKFLOW
```

```
def run_text_to_sql():
```

```
    print("\n=====")
```

```
    print("    TEXT-TO-SQL WORKFLOW")
```

```
    print("=====")
```

```
# STEP 0: SETUP DATABASE
```

```
print("\n[0] Setting up database...")
```

```
setup_database()
```

```
print("Database ready.")

# STEP 1: GET SCHEMA

print("\n[1] Retrieving database schema...")

schema = get_schema()

print("\nDatabase Schema:")
print(schema)

# STEP 2: USER QUESTION

question = input(
    "\nEnter your question about the database: "
)

# STEP 3: GENERATE SQL

print("\n[2] Generating SQL query...")

sql = generate_sql(
    question,
    schema
)

print("\nGenerated SQL:")
print(sql)

# STEP 4: VALIDATE SQL

print("\n[3] Validating SQL query...")

if not validate_sql(sql):

    print("\nInvalid SQL query.")
    print("Only SELECT queries are allowed.")

    return

print("SQL query is valid.")

# STEP 5: EXECUTE SQL
```

```

print("\n[4] Executing SQL query...")

columns, results = execute_sql(sql)

if columns is None:

    print("\nSQL execution error:")
    print(results)

    return

print("\nQuery Results:")

print(columns)

for row in results:
    print(row)

# STEP 6: GENERATE ANSWER

print("\n[5] Generating final answer...")

answer = generate_answer(
    question,
    columns,
    results
)

print("\nFinal Answer:")
print(answer)

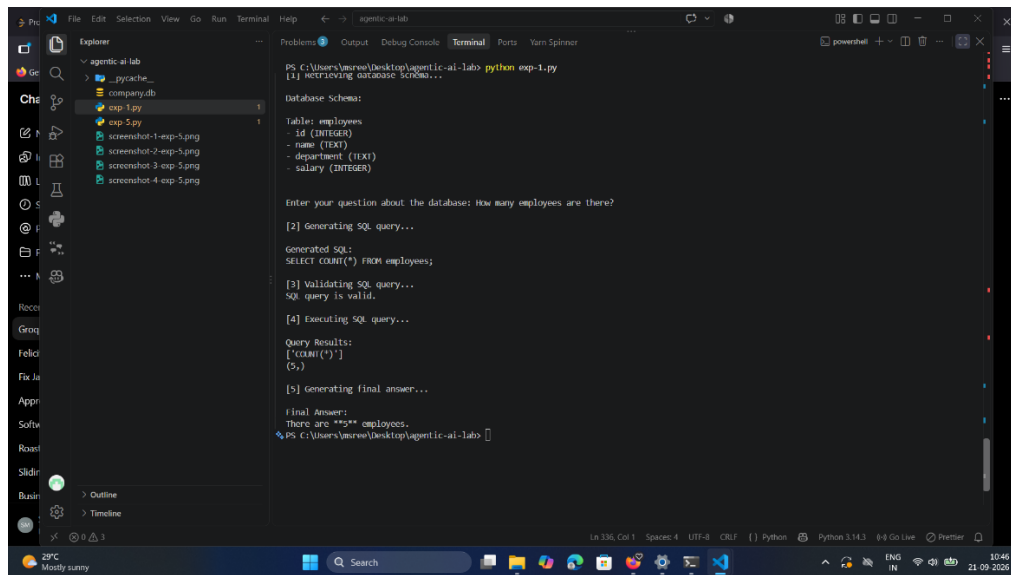
# PROGRAM START

if __name__ == "__main__":

    run_text_to_sql()

```

Output :-



3. Lab-exp-6 :- Policy Compliance Agent

Develop an AI-based policy compliance agent that evaluates employee data against predefined organizational security policies, identifies compliance violations, and generates a compliance report with recommended remediation actions.

Code :-

```
from groq import Groq
```

```
# GROQ API KEY
```

```
API_KEY = "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"
```

```
client = Groq(api_key=API_KEY)
```

```
# LLM FUNCTION
```

```
def ask_llm(prompt):
```

```

    response = client.chat.completions.create(
        model="openai/gpt-oss-120b",
        messages=[
            {
                "role": "user",
                "content": prompt
            }
        ]
    )

```

```

    ]
)

return response.choices[0].message.content

# SYNTHETIC EMPLOYEE DATA

employees = [
    {
        "name": "Rahul",
        "department": "Engineering",
        "mfa_enabled": True,
        "password_shared": False,
        "sensitive_data_personal_device": False
    },
    {
        "name": "Priya",
        "department": "HR",
        "mfa_enabled": False,
        "password_shared": False,
        "sensitive_data_personal_device": True
    },
    {
        "name": "Arjun",
        "department": "Finance",
        "mfa_enabled": True,
        "password_shared": True,
        "sensitive_data_personal_device": False
    },
    {
        "name": "Sneha",
        "department": "Marketing",
        "mfa_enabled": True,
        "password_shared": False,
        "sensitive_data_personal_device": False
    },
    {
        "name": "Vikram",
        "department": "Engineering",
        "mfa_enabled": False,
        "password_shared": True,
        "sensitive_data_personal_device": True
    }
]

```

```
# COMPLIANCE RULES
```

```
def check_compliance(employee):
```

```
    violations = []
```

```
    # Rule 1: MFA must be enabled
```

```
    if not employee["mfa_enabled"]:
```

```
        violations.append(
            "MFA is not enabled."
        )
```

```
    # Rule 2: Passwords must not be shared
```

```
    if employee["password_shared"]:
```

```
        violations.append(
            "Password sharing detected."
        )
```

```
    # Rule 3: Sensitive data must not be stored
```

```
    # on personal devices
```

```
    if employee["sensitive_data_personal_device"]:
```

```
        violations.append(
            "Sensitive data is stored on a personal device."
        )
```

```
    # Determine status
```

```
    if len(violations) == 0:
```

```
        status = "COMPLIANT"
```

```
    else:
```

```
        status = "NON-COMPLIANT"
```

```
    return {
```

```
        "name": employee["name"],
        "department": employee["department"],
        "status": status,
        "violations": violations
    }
```

```

# RUN COMPLIANCE CHECK

def evaluate_employees():

    results = []

    for employee in employees:

        result = check_compliance(employee)

        results.append(result)

    return results

# GENERATE COMPLIANCE REPORT

def generate_report(results):

    report_data = ""

    for result in results:

        report_data += f"""
Employee: {result["name"]}
Department: {result["department"]}
Status: {result["status"]}
Violations: {result["violations"]}
-----
"""

    prompt = f"""
You are a Policy Compliance Reporting Agent.

Analyze the following compliance evaluation results.

Compliance Results:
{report_data}

Create a professional compliance report.

The report must contain:

1. Executive Summary

```

2. Total Employees Evaluated
3. Number of Compliant Employees
4. Number of Non-Compliant Employees
5. Employee-wise Findings
6. Recommended Remediation Actions

Important:

- Use only the information provided.
- Do not invent violations.
- Clearly distinguish compliant and non-compliant employees.
- Keep the report concise and professional.

"""

```
    return ask_llm(prompt)
```

```
# MAIN POLICY COMPLIANCE AGENT
```

```
def run_compliance_agent():
```

```
    print("\n=====")
    print("        POLICY COMPLIANCE AGENT")
    print("=====")
```

```
# STEP 1: EVALUATE EMPLOYEES
```

```
print("\n[1] Evaluating employee compliance...")
```

```
results = evaluate_employees()
```

```
# STEP 2: DISPLAY RULE-BASED RESULTS
```

```
print("\n=====")
print("        COMPLIANCE CHECK RESULTS")
print("=====")
```

```
for result in results:
```

```
    print(f"\nEmployee: {result['name']}")
    print(f"Department: {result['department']}")
    print(f"Status: {result['status']}")
```

```
    if result["violations"]:
```

```

        print("Violations:")

        for violation in result["violations"]:

            print(f" - {violation}")

    else:

        print("Violations: None")

# STEP 3: GENERATE AI REPORT

print("\n=====")
print("      AI COMPLIANCE REPORT")
print("=====")

report = generate_report(results)

print("\n")
print(report)

# PROGRAM START

if __name__ == "__main__":

    run_compliance_agent()

```

Output :-

